

Рекурсивные функции

В данном уроке мы узнаем, что такое рекурсия и как ее использовать.

До сих пор мы рассматривали итеративные функции. То есть функции выполнялись последовательно и друг за другом. Но что, если у нас повторяется одно и то же действие последовательно несколько раз? Рассмотрим пример. Мы хотим найти сумму натуральных чисел на определенном промежутке, например, от 1 до 5, то есть $1+2+3+4+5=15$. На python код мог бы выглядеть следующим образом:

```
def summa(n):  
    x = 0  
    for n in range(1, n+1):  
        x += n  
    return x  
  
summa(5)  
>>> 15
```

Как видите, мы написали цикл, в котором $n+1$ раз прибавляли последовательно каждое последующее число.

Но что, если попробовать написать функцию, которая бы осуществляла сложение с предыдущим элементом, а потом бы вызывала саму себя несколько раз? Такой способ программирования называется рекурсией, а функция, соответственно, рекурсивной. Используя рекурсию, наш код можно будет переписать следующим образом:

```
def summa(n):  
    if n == 1:  
        return 1  
    return n + summa(n-1)  
  
summa(5)  
>>> 15
```

Во втором варианте задача была решена с использованием её же самой.

$\text{summa}(5)$ — то же самое, что $5 + \text{summa}(4)$

$\text{summa}(4)$ — то же самое, что $4 + \text{summa}(3)$

$\text{summa}(3)$ — то же самое, что $3 + \text{summa}(2)$

$\text{summa}(2)$ — то же самое, что $2 + \text{summa}(1)$

$\text{summa}(1)$ — это 1

Условия рекурсивных алгоритмов

Вернёмся к нашему примеру с `summa(n)`. Чтобы алгоритм работал, программа должна соответствовать двум требованиям.

Базовый случай

Помимо рекурсивного случая, когда функция вызывает сама себя ещё раз, должно быть определённое стоп-условие, чтобы этот процесс не продолжался бесконечно и функция могла завершить работу самостоятельно. В программировании это называется базовым случаем.

У нас он произойдёт, когда `n` станет равным 1. Мы упростили задачу настолько, что больше нет смысла считать, — и можем просто дать ответ.

Рекурсия

Чтобы прийти к базовому случаю, мы должны передавать каждой новой функции изменённые данные. Другими словами, нужно изменять аргумент функции.

В нашем коде при первом вызове `n` была равна 5, при втором — 4. И так до тех пор, пока `n` не стала равна 1. Не сделай мы этого, рекурсия никогда бы не завершилась и бесконечно порождала бы новые функции.

Рекурсивно или итеративно?

Каковы же преимущества рекурсивных функций? Можно ли с помощью итеративных получить тот же результат? Когда лучше использовать одни, а когда — другие?

Важно учитывать временную и пространственную сложности. Рекурсивные функции занимают больше места в памяти по сравнению с итеративными из-за постоянного добавления новых слоев в стек в памяти. Однако их производительность куда выше.

Рекурсия может быть медленной, если реализована неправильно

Тем не менее рекурсия может быть медленной, если ее неправильно реализовать. Из-за этого вычисления будут происходить чаще, чем требуется.

Написание итеративных функций зачастую требуется большего количества кода.

Задание.

1. Реализовать функцию нахождения факториала с использованием рекурсии и без. Сравнить эффективность каждого решения.
2. Реализовать функцию возведения в квадрат чисел из списка. На вход подается список чисел, на выход должен выдаваться список значений в квадрате.